

Data Wrangling: Cleaning, Merging & GroupBy



Suman

22 Feb, 2026 At 8:00 PM

Notes

Module-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 4.2: Lecture Notes

Colab File: [Click Here](#)

PPT File: [Click Here](#)

Data Wrangling: Cleaning, Merging & GroupBy

Program: AIM101 -- Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Session Type:** Instructor (Practical, Hands-on) **Prerequisites:** Session 4.1 (Pandas DataFrames: Creation, Indexing & Selection)

Mental Map

```
Session 4.2: Data Wrangling – Cleaning, Merging & GroupBy
|
+-- A. HANDLING MISSING VALUES
|   +-- What is NaN? (np.nan, None -> NaN)
|   +-- Detecting: .isna(), .notna(), .isna().sum()
|   +-- .info() Non-Null Count as first check
|   +-- Dropping: .dropna(subset=, thresh=)
|   +-- Filling: .fillna(value), .fillna(mean/median), .ffill()
|   +-- Decision framework: when to drop vs when to fill
|
+-- B. HANDLING DUPLICATES
|   +-- Why duplicates are dangerous (silent inflation)
|   +-- Detecting: .duplicated(), .duplicated(subset=, keep=)
|   +-- Removing: .drop_duplicates(subset=, keep=)
|   +-- The subset parameter: unique identifier columns
|
+-- C. DATA TYPE CONVERSION
|   +-- The hidden type problem (object dtype for numbers/dates)
|   +-- .astype() – simple, direct casting (clean data only)
```

```

|   +-- pd.to_numeric(errors='coerce') - safe numeric conversion
|   +-- pd.to_datetime(format='mixed') - safe date conversion
|   +-- errors parameter: 'raise' vs 'coerce' vs 'ignore'
|
+-- D. STRING OPERATIONS (.str accessor)
|   +-- .str.strip() - remove whitespace
|   +-- .str.lower(), .str.upper(), .str.title() - case standardization
|   +-- .str.replace(), .str.contains() - search and replace
|   +-- .str.split(), .str.len() - structure and validation
|   +-- 3-step pipeline: strip -> case -> specific fixes
|
+-- E. MESSY DATA PATTERNS
|   +-- Tidy data: 3 rules (Wickham, 2014)
|   +-- Wide format vs Long format
|   +-- pd.melt() - wide to long (unpivot)
|   +-- .pivot() - long to wide
|   +-- Pattern recognition (awareness level)
|
+-- F. COMBINING DATAFRAMES: CONCAT
|   +-- pd.concat() - "dumb gluing" (no key matching)
|   +-- Vertical stacking (axis=0): more rows, same columns
|   +-- Horizontal stacking (axis=1): same rows, more columns
|   +-- ignore_index=True - prevent duplicate indices
|   +-- join='inner' vs join='outer' for mismatched columns
|
+-- G. COMBINING DATAFRAMES: MERGE/JOIN
|   +-- pd.merge() - VLOOKUP on steroids
|   +-- on=, left_on=, right_on= for key columns
|   +-- INNER join: only matching rows (default)
|   +-- LEFT join: all left rows + matching right (most common)
|   +-- RIGHT join: all right rows + matching left
|   +-- OUTER join: everything from both sides
|   +-- indicator=True for debugging merges
|   +-- Many-to-many danger: Cartesian product / row explosion
|
+-- H. GROUPBY & AGGREGATION
|   +-- Split-Apply-Combine pattern
|   +-- .groupby('col')['val'].sum/mean/count()
|   +-- .agg() - multiple aggregations, named aggregations
|   +-- .transform() - broadcast group values back to rows
|   +-- .pivot_table() - cross-tabulation summaries
|   +-- .reset_index() - always call after groupby

```

The Big Picture

The Data Wrangling Pipeline

+-----+		+-----+		+-----+		+-----+
DETECT	-->	CLEAN	-->	COMBINE	-->	ANALYZE
.info()		.fillna()		pd.concat()		.groupby()
.isna().sum()		.dropna()		pd.merge()		.agg()
.duplicated()		.drop_duplicates		(4 join types)		.transform()
.dtypes		pd.to_numeric()				.pivot_table

.describe()	pd.to_datetime()		
	.str.strip()		
	.str.lower()		
+-----+	+-----+	+-----+	+-----+

Session 4.1 vs Session 4.2

Session 4.1	Session 4.2
Create DataFrames	Clean messy DataFrames
Load CSV files	Fix data types, strings, missing values
Select, filter, sort	Combine multiple DataFrames
Work with clean data	Work with real-world messy data
Single DataFrame operations	Multi-DataFrame pipelines
Data Consumer	Data Wrangler

Key Insight: Data scientists spend 60-80% of their time on data wrangling. Everything in this session is what fills that 80%.

TOPIC A: Handling Missing Values

A.1 What Is NaN?

- **NaN** = Not a Number -- Pandas' universal marker for "this value is missing"
- NaN comes from NumPy: `np.nan`
- NaN is NOT zero. NaN is NOT an empty string. NaN is NOT False.
- NaN is the **absence** of a value

The strangest property:

```
np.nan == np.nan    # Returns False!
```

NaN in calculations:

- `5 + NaN = NaN` (any math with NaN produces NaN -- the "poison" property)
- Pandas `.mean()`, `.sum()` **ignore NaN by default** (unlike raw NumPy)

Where NaN appears:

- Loading a CSV with empty cells
- Merging DataFrames with mismatched keys

- Creating DataFrames from dictionaries with unequal-length lists
- `None` in a numeric column auto-converts to NaN

FORM ANALOGY:

```

+-----+
| Patient Intake Form |
+-----+
| Name:    Rajesh Kumar | <- filled
| Age:     45           | <- filled
| Blood:   A+          | <- filled
| Allergy: _____ | <- BLANK = NaN
| Weight:  _____ | <- BLANK = NaN
+-----+
Blank != "None"   Blank != "0"   Blank = "Not recorded"

```

A.2 Detecting Missing Values

Step 1: Bird's-eye view with `.info()`

```

df.info()
# Look at the "Non-Null Count" column
# If a column has 10 entries but only 8 non-null -> 2 missing values

```

Step 2: Boolean mask with `.isna()`

```

df['age'].isna()      # Series of True/False for one column
df.isna()            # DataFrame of True/False for all columns

```

Step 3: Count missing values

```

# The Data Quality Snapshot -- memorize this pattern
df.isna().sum()          # Count of missing per column
df.isna().sum().sum()    # Total missing in entire DataFrame
df.isna().mean() * 100   # Percentage missing per column

```

Inverse: `.notna()` returns True where values ARE present

```

df[df['age'].notna()]    # Only rows where age exists

```

A.3 Dropping Missing Values -- `.dropna()`

Syntax	Behavior
<code>df.dropna()</code>	Drop any row with ANY NaN (aggressive!)
<code>df.dropna(subset=['age'])</code>	Drop rows only where 'age' is NaN (surgical)

Syntax	Behavior
<code>df.dropna(thresh=5)</code>	Keep rows with at least 5 non-null values
<code>df.dropna(axis=1)</code>	Drop COLUMNS with any NaN (rarely used)

Key behavior: `.dropna()` returns a NEW DataFrame -- the original is unchanged.

Pitfall: Blind `df.dropna()` can lose half your data. Always check the shape before and after.

A.4 Filling Missing Values -- `.fillna()`

Strategy	Code	When to Use
Fill with constant	<code>df['name'].fillna('Unknown')</code>	Placeholder for text columns
Fill with mean	<code>df['age'].fillna(df['age'].mean())</code>	Numerical data, symmetric distribution
Fill with median	<code>df['age'].fillna(df['age'].median())</code>	Numerical data, robust to outliers
Forward fill	<code>df['temp'].ffill()</code>	Time series (use previous value)
Backward fill	<code>df['temp'].bfill()</code>	Time series (use next value)

A.5 Decision Framework: Drop vs Fill

```

IS THE COLUMN CRITICAL?
+-- YES (e.g., patient blood type)
|   +-- Can you look it up from another source?
|       +-- YES -> Look it up and fill
|       +-- NO  -> Flag the row, do NOT guess
|
+-- NO (e.g., middle name)
    +-- What % is missing?
        +-- < 5%  -> DROP rows with .dropna(subset=[...])
        +-- 5-30% -> FILL with mean/median/mode
        +-- > 50% -> DROP the entire COLUMN

```

There is NO universal correct answer. The right strategy depends on the domain, the column's importance, and the percentage missing.

KEY FRAMEWORKS

Drop vs Fill Decision Framework

Scenario	Recommended Action
Column is critical (blood type) + cannot look up	Flag the row -- do NOT guess
Column is critical + can look up from another source	Fill from the source
Column is non-critical + <5% missing	Drop rows with <code>.dropna(subset=[...])</code>
Column is non-critical + 5-30% missing	Fill with mean/median/mode
Column has >50% missing	Drop the entire column

Which Join Type to Use

Situation	Join Type	Why
Only want matched records	<code>inner</code>	Strict -- no NaN in result
Enrich primary table with lookups	<code>left</code>	Most common -- keep all primary rows
Find unmatched records (debug)	<code>outer + indicator=True</code>	See what is missing on each side
Reverse perspective of left join	<code>right</code>	Swap arguments and use <code>left</code> instead

concat vs merge

	<code>pd.concat()</code>	<code>pd.merge()</code>
Logic	Dumb gluing -- stacks DataFrames	Smart lookup -- matches on key columns
Use when	Same structure, different data batches	Different tables with a shared key
Analogy	Stacking cafeteria trays	VLOOKUP in Excel
Key matching	None -- just stacks	Yes -- uses <code>on=</code> parameter

.agg() vs .transform() Quick Decision

Question	Use
"What is the total sales per city?" (summary)	<code>.agg()</code>
"What % of city sales is each order?" (context)	<code>.transform()</code>
Need fewer rows (one per group)	<code>.agg()</code>

Question	Use
Need same rows with group info added	<code>.transform()</code>

COMMON ERRORS QUICK REFERENCE

#	Error	Cause	Fix
1	<code>ValueError: Unable to parse string "N/A"</code>	<code>pd.to_numeric()</code> with default <code>errors='raise'</code>	Use <code>errors='coerce'</code>
2	String concatenation instead of addition	Column stored as <code>object</code> (string), not numeric	Convert with <code>pd.to_numeric(errors='coerce')</code> first
3	<code>'Cardiology'</code> and <code>'cardiology'</code> counted separately	Inconsistent casing in text column	<code>df['col'] = df['col'].str.lower()</code>
4	<code>.duplicated()</code> returns 0 but duplicates exist	Whitespace difference hides duplicates	Use <code>subset=['id_column']</code> instead of checking all columns
5	Duplicate index values after <code>pd.concat()</code>	Forgot <code>ignore_index=True</code>	<code>pd.concat([df1, df2], ignore_index=True)</code>
6	Row explosion after <code>pd.merge()</code>	Many-to-many merge (non-unique key in both tables)	Check <code>df['key'].is_unique</code> before merging
7	GroupBy result has grouped column as index	Did not call <code>.reset_index()</code>	Chain <code>.reset_index()</code> after <code>.agg()</code>
8	<code>NaN</code> forced integer column to float64	Pandas upcasts int to float when NaN is present	Expected behavior; use <code>Int64</code> nullable type if needed
9	<code>.fillna()</code> or <code>.dropna()</code> did not change the DataFrame	These methods return NEW DataFrames	Assign: <code>df = df.dropna()</code> or <code>df['col'] = df['col'].fillna(0)</code>
10	<code>pd.to_datetime()</code> parsing wrong dates	Ambiguous dates (01-02-2024: Jan 2 or Feb 1?)	Use <code>dayfirst=True</code> for DD-MM-YYYY format
11	<code>.str</code> method crashed with <code>AttributeError</code>	Column contains non-string types	Ensure column is <code>object</code> dtype; <code>.str</code> only works on strings

#	Error	Cause	Fix
12	<code>merge()</code> produced unexpected NaN columns	Column name exists in both DataFrames	Use <code>suffixes=('_left', '_right')</code> parameter
13	<code>pivot_table()</code> shows NaN for missing combinations	No data for that row-column combination	Use <code>fill_value=0</code> parameter

KEY TAKEAWAYS

MISSING DATA DETECTION: Always start with `.info()` and `.isna().sum()` to understand what is missing and where. The percentage of missing values per column determines your handling strategy.

MISSING DATA HANDLING: Use `.fillna()` for imputation (mean, median, or domain-appropriate constant) and `.dropna()` for removal. Choose based on context -- a missing blood type cannot be "averaged," but a missing weight can.

DUPLICATE REMOVAL: Use `.duplicated(subset=['id_column'])` to detect and `.drop_duplicates()` to remove. Always investigate duplicates before blindly removing them -- whitespace and casing differences can hide obvious duplicates.

DATA TYPE CONVERSION: Use `pd.to_numeric(errors='coerce')` for safe numeric conversion and `pd.to_datetime(format='mixed')` for dates. The `errors='coerce'` pattern converts what it can and turns failures into NaN for later review.

STRING CLEANING: The `.str` accessor applies string methods to entire columns at once. The 3-step pipeline (strip whitespace, standardize case, fix specifics) handles 90% of text data quality issues. Inconsistent casing is an invisible data quality killer.

CONCATENATION: `pd.concat()` stacks DataFrames vertically (more rows) or horizontally (more columns). Always use `ignore_index=True` to avoid duplicate index values.

MERGING / JOINS: `pd.merge()` combines DataFrames on shared key columns using four join types: inner (intersection), left (all primary + matched lookup), right (all lookup + matched primary), and outer (everything). Left join is the most common in practice.

GROUPBY + AGG: The split-apply-combine pattern (`.groupby().agg().reset_index()`) is the most powerful single operation in Pandas for summarization. Named aggregations produce the cleanest output.

PIVOT TABLES & TRANSFORM: `.pivot_table()` creates cross-tabulation summaries (rows x columns). `.transform()` broadcasts group-level values back to individual rows, enabling relative analysis like "what percentage of city sales does this order represent?"

NEXT SESSION PREVIEW

Session 4.3: Data Quality -- Principles & Industry Horror Stories

Topic	What You Will Learn
Data Quality Dimensions	Accuracy, Completeness, Consistency, Timeliness
Garbage In, Garbage Out	Even the best ML model fails on bad data
Industry Horror Stories	IBM Watson Oncology, JPMorgan London Whale, UK Post Office Horizon
Data Governance Basics	Documentation, validation, regulatory awareness

Why it matters:

- This session you learned HOW to clean data
- Next session you learn WHY data quality is the difference between a working model and a billion-dollar disaster

Key cases:

- **IBM Watson Oncology:** AI gave unsafe cancer treatment recommendations because training data was based on hypothetical cases, not real patients
- **JPMorgan London Whale:** \$6 BILLION loss partly caused by a copy-paste error in a spreadsheet risk model
- **UK Post Office Horizon:** Faulty software data sent 700+ innocent sub-postmasters to prison over 20 years

Note: Session 4.3 is a Professor session -- NO CODE. Pure theory, case studies, and frameworks.

Footer: Vishlesan i-Hub IIT Patna x Masai School